

RinSathi Mid-Defense Engineering Handbook

Complete codebase guide, defense preparation manual, and professional explanation bank.

Project: RinSathi - Autonomous Credit & Lending Orchestrator

Audience: RinSathi team members preparing for mid-defense

Purpose: One document that explains what the project does, how every folder/file contributes, what important code means, how to demo Swagger, and how to answer questions like professional engineers.

Codebase reviewed: June 16, 2026

Table of Contents

1. What RinSathi Is
2. Mid-Defense Pitch
3. Complete System Flow
4. Folder-by-Folder Map
5. File-by-File Deep Explanation With Code Concepts
6. Swagger Docs: What To Show And What To Say
7. Important Engineering Concepts In Layman Terms
8. Known Limitations And Honest Professional Answers
9. Live Demo Script
10. Defense Q&A Bank
11. Team Study Plan
12. Final Professional Closing

1. What RinSathi Is

RinSathi is an AI-assisted loan assessment system designed for microfinance and rural lending in Nepal. Its main job is to take a loan application, read the applicant's documents, estimate income from alternative data sources, calculate a repayment score, check compliance rules, and return a final recommendation.

In normal banking, a person without a salary slip or formal bank statement may be rejected even if they have real income. RinSathi tries to solve that by using alternative signals such as eSewa-like wallet transactions, remittance records, cooperative savings records, and identity documents.

The system is built around five agents:

1. Document Agent - reads identity/land documents using OCR.

2. Income Agent - calculates monthly income from alternative financial records.
3. Score Agent - uses a machine learning model to estimate repayment strength.
4. Compliance Agent - checks rules and regulatory guardrails.
5. Decision Agent - gives final decision and explanation.

Defense line:

"RinSathi is a FastAPI-based, multi-agent credit assessment pipeline that combines OCR, alternative income analysis, machine learning scoring, compliance rules, and explainability to support microfinance loan officers."

Important safety line:

"RinSathi is a decision-support system. It helps loan officers make faster and more auditable decisions; it does not blindly replace all human judgment."

2. Mid-Defense Pitch

Use this when the examiner asks, "Explain your project."

"Our project, RinSathi, addresses a major problem in rural microfinance: many applicants do not have formal salary slips, bank statements, or credit history, but they still have real repayment capacity through remittances, mobile wallet income, cooperative deposits, and agricultural income. We built a five-agent AI-assisted pipeline. The Document Agent extracts document data using PaddleOCR and OpenCV. The Income Agent normalizes alternative cashflow sources into a monthly income estimate. The Score Agent uses an XGBoost model to calculate repayment probability. The Compliance Agent checks rule-based constraints such as KYC quality, income reliability, loan-to-asset risk, sector exposure, and AML-style flags. Finally, the Decision Agent produces a recommendation, referral, or rejection reason. We expose the system through FastAPI, a dashboard frontend, and Swagger API documentation."

Short version:

"RinSathi turns informal borrower data into a structured, explainable, compliance-aware loan recommendation."

3. Complete System Flow

3.1 User Flow

1. User opens the portal at /.
2. User goes to the underwriting dashboard at /dashboard.

3. User enters requested loan amount and business sector.
4. User uploads a citizenship or Lalpurja image.
5. User enables mock income data for demo, or uses income console at /apply.
6. Frontend sends the request to POST /api/v1/loan/apply.
7. Backend creates a SharedState object for the application.
8. Document Agent and Income Agent run concurrently.
9. Score Agent calculates score.
10. Compliance Agent checks rule flags.
11. Decision Agent generates verdict.
12. API returns JSON.
13. Dashboard displays result.

3.2 Technical Flow

The most important internal object is SharedState. Think of it as a loan application file folder. At the beginning, it contains applicant ID, loan amount, and sector. Each agent writes its output into this same folder.

Flow:

Frontend form

- > FastAPI route /api/v1/loan/apply
- > SharedState created
- > Document Agent + Income Agent run together
- > Score Agent
- > Compliance Agent
- > Decision Agent
- > JSON response
- > Dashboard display

Defense line:

"We used a SharedState pattern because it keeps the pipeline simple. Every agent has one input and one output: it receives the state, updates its fields, and returns the state."

4. Folder-by-Folder Map

Root folder

The root contains app startup files, configuration, documentation, test helpers, and generated assets.

Important root files:

- main.py: FastAPI app entrypoint.
- config.py: central configuration from .env.
- requirements.txt: dependency list.
- README.md: public overview.
- mock_defense_guide.md: earlier defense notes.
- alembic.ini: database migration configuration.
- test_*.py and test_score.html: development testing helpers.
- fix_*.py: temporary repair/helper scripts created during development.

`'agents/'`

Core business logic. This is where the multi-agent pipeline lives.

`'api/'`

HTTP layer. It defines request/response models and API endpoints.

`'db/'`

Database models, database connection, and migration setup.

`'frontend/templates/'`

HTML pages served by FastAPI.

`'utils/'`

Reusable helper functions for OCR, income parsing, and explanation formatting.

`'ml/'`

Machine learning training script and saved trained model.

`'docs/'`

Defense handbook, reference PDF, and PDF generator.

`'venv/'`

Python virtual environment. It contains installed packages. Do not explain every file inside it; it is generated by Python tooling.

‘__pycache__/'

Compiled Python cache files. These are generated automatically and are not handwritten source code.

‘.git/'

Git version-control database. Not part of runtime logic.

5. File-by-File Deep Explanation With Code Concepts

This is the most important section for your team. If an examiner opens any file, use the matching explanation.

5.1 ‘main.py’

What This File Does

main.py is the starting point of the backend. When we run:

```
uvicorn main:app --reload
```

Uvicorn imports the app object from main.py and starts the FastAPI server.

Main Code Portion

```
app = FastAPI(  
    title="RinSathi ACLO API",  
    description="Autonomous Credit and Lending Orchestrator ...",  
    version="1.0.0",  
    lifespan=lifespan,  
)
```

Layman explanation:

This creates the web application. The title, description, and version also appear in Swagger docs. That means main.py does not only start the server; it also defines the identity of the API documentation.

Startup Lifecycle

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await create_tables()
    from utils.ocr import warm_up_ocr
    await warm_up_ocr()
    yield
```

Layman explanation:

Before the server starts accepting requests, it prepares the database tables and warms up the OCR engine. Warming OCR means loading its model earlier so the first real request is not too slow.

Defense line:

"We use FastAPI lifespan to run startup tasks once: table creation and OCR warmup. This improves reliability and first-request performance."

Middleware

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Layman explanation:

CORS allows browser frontend pages to call the backend API. During demo/development, it allows all origins for convenience.

Professional caveat:

"In production, we would restrict allow_origins to the official frontend domain."

Router Registration

```
app.include_router(documents.router, prefix="/api/v1")
app.include_router(income.router, prefix="/api/v1")
app.include_router(loan.router, prefix="/api/v1")
```

Layman explanation:

This connects the route files to the main app. The prefix="/api/v1" means endpoints become versioned paths like /api/v1/loan/apply.

Frontend Routes

```
@app.get("/")
async def root(request: Request):
    return templates.TemplateResponse(request=request, name="login.html")
```

This serves the login page. Similar routes serve /apply and /dashboard.

Health Check

```
@app.get("/health", tags=["System"])
async def health():
    return {
        "status": "ok",
        "version": "1.0.0",
        "ocr_mode": "paddleocr" if OCR_AVAILABLE else "mock",
        "database": "supabase",
    }
```

Layman explanation:

This is a quick endpoint to check whether the system is alive and whether OCR is running in real or fallback mode.

Important Defense Note

There is also a POST /api/v1/income/analyze route inside main.py, while another route with the same path exists in api/routes/income.py. This is a mid-defense integration cleanup item.

Professional answer:

"The app currently has duplicate income-analysis route definitions from sprint integration. The intended production cleanup is to keep the route inside api/routes/income.py and remove the controller duplicate from main.py."

5.2 'config.py'

What This File Does

config.py stores application settings in one place. Instead of hardcoding thresholds across the

project, the system reads values from .env.

Main Code Portion

```
class Settings(BaseSettings):
    DATABASE_URL: str
    SECRET_KEY: str = "change-in-production"
    APPROVE_THRESHOLD: float = 0.65
    REFER_THRESHOLD: float = 0.40
    MIN_KYC_CONFIDENCE: float = 0.70
    MAX_LOAN_TO_ASSET: float = 0.75
    AML_TXN_LIMIT_NPR: float = 1_000_000
    AGRI_SECTOR_LIMIT_NPR: float = 500_000
    OCR_TIMEOUT_SECONDS: float = 10.0
```

Layman explanation:

These are project control knobs. If NRB rules or project thresholds change, we should change them in .env/config instead of editing five different files.

Cached Settings

```
@lru_cache()
def get_settings() -> Settings:
    return Settings()
```

Layman explanation:

The settings are loaded once and reused. This avoids repeatedly reading .env.

Defense line:

"This follows the Twelve-Factor App principle: configuration lives outside business logic, so the same code can run in development, demo, and production with different settings."

5.3 '.env'

What This File Does

.env stores environment-specific secrets and settings such as database URL and secret key.

Do not show real secret values in defense.

Defense line:

"The .env file is intentionally local and should not be committed with real production secrets. It allows us to configure the app without hardcoding credentials."

5.4 'requirements.txt'

What This File Does

Lists installed Python packages needed for the project.

Important packages:

- fastapi: API framework.
- uvicorn: server runner.
- pydantic and pydantic-settings: validation and config.
- SQLAlchemy and asyncpg: database access.
- paddleocr and paddlepaddle: OCR engine.
- opencv-python-headless: image preprocessing.
- pandas, numpy: data processing.
- xgboost, scikit-learn: ML training and inference.
- joblib: model saving/loading.
- python-multipart: file uploads in FastAPI.
- Jinja2: templates.

Defense line:

"The requirements file makes the environment reproducible. Anyone can install the same dependencies with `pip install -r requirements.txt`."

5.5 'README.md'

What This File Does

Provides the public-facing explanation of the project: problem, architecture, tech stack, setup, and usage.

Defense line:

"README is for onboarding developers and evaluators. The handbook is deeper; README is the quick start."

5.6 'mock_defense_guide.md'

What This File Does

Earlier defense guide with architecture explanations and sample answers. This new handbook expands it with file-by-file code explanation.

Defense line:

"This file was an earlier preparation note. We upgraded it into the full mid-defense handbook."

5.7 'alembic.ini'

What This File Does

Alembic configuration for database migrations.

Important line:

```
script_location = %(here)s/db/migration
```

Layman explanation:

This tells Alembic where migration scripts live.

Defense line:

"Alembic helps us version database schema changes safely instead of manually editing tables."

5.8 'api/__init__.py', 'agents/__init__.py', 'db/__init__.py', 'api/routes/__init__.py', 'utils/__init__.py'

What These Files Do

These files mark folders as Python packages. They may be empty, but they are useful because Python can import modules from those folders cleanly.

Defense line:

"The __init__.py files organize our code into importable packages like agents, api, db, and utils."

5.9 'api/schemas.py'

What This File Does

Defines API response shapes using Pydantic models.

Main Code Portion

```
class FieldResult(BaseModel):  
    value: str  
    confidence: float
```

Layman explanation:

One OCR field has a text value and a confidence score.

```
class DocumentUploadResponse(BaseModel):  
    applicant_id: uuid.UUID  
    document_verified: bool  
    extracted_fields: dict[str, FieldResult]  
    doc_confidence: float  
    manual_review_required: bool  
    processing_time_ms: int  
    ocr_mode: str
```

Layman explanation:

This is the expected response format after document processing. It tells the frontend what fields to expect.

Swagger connection:

FastAPI reads these models and automatically shows response schemas in /docs.

Defense line:

"Pydantic schemas are our API contract. They make the API predictable for frontend developers and generate Swagger documentation automatically."

5.10 'api/routes/documents.py'

What This File Does

Handles document upload and OCR polling.

Main Code Portion: In-Memory Job Store

```
jobs_db: Dict[str, dict] = {}
```

Layman explanation:

This is a temporary memory-based tracking table. When a document is uploaded, the system creates a job ID and stores job status here.

Professional caveat:

"For production, this should move from memory to Redis or database storage because memory resets when the server restarts."

Background Task

```
async def ocr_worker_task(job_id: str, image_bytes: bytes):  
    jobs_db[job_id]["status"] = "processing"  
    ocr_result = await run_ocr(image_bytes)  
    final_payload = extract_fields(ocr_result)  
    jobs_db[job_id]["status"] = "completed"  
    jobs_db[job_id]["result"] = final_payload
```

Layman explanation:

The server accepts the file quickly, then OCR runs in the background. This prevents the request from waiting too long.

Upload Endpoint

```
@router.post("/document/upload")  
async def upload_document(background_tasks: BackgroundTasks, file: UploadFile =  
File(...)):  
    image_bytes = await file.read()  
    job_id = str(uuid.uuid4())  
    background_tasks.add_task(ocr_worker_task, job_id, image_bytes)
```

Layman explanation:

The API reads the uploaded file, creates a unique tracking ticket, and starts OCR after returning a pending response.

Polling Endpoint

```
@router.get("/document/result/{job_id}")
async def get_ocr_result(job_id: str):
    if job_id not in jobs_db:
        raise HTTPException(status_code=404, detail="Job ID tracking ticket not found.")
```

Layman explanation:

The frontend can keep asking, "Is my OCR finished?" using the job ID.

Defense line:

"This uses a ticket-polling pattern. It is useful for slow OCR jobs because upload and processing are decoupled."

5.11 'api/routes/income.py'

What This File Does

Exposes income-analysis endpoints.

Request Schema

```
class IncomeAnalyzeRequest(BaseModel):
    applicant_id: uuid.UUID
    esewa_data: Optional[dict] = None
    remittance_data: Optional[dict] = None
    coop_data: Optional[dict] = None
    use_mock_data: bool = False
```

Layman explanation:

The user can provide any of the three income data types. During demo, use_mock_data=True generates sample data automatically.

Validation

```
if not any([esewa_data, remittance_data, coop_data]):
    raise HTTPException(status_code=422, detail="At least one income source is
```

required...")

Layman explanation:

The API refuses to analyze income if no income data is provided.

Income Parsing

```
sigs, name = parse_esewa(esewa_data)
all_signals.extend(sigs)
```

Layman explanation:

Each parser converts its source into a common signal format. Then all signals are combined.

Response

```
return IncomeAnalyzeResponse(
    mean_monthly_npr=estimate["mean_monthly_npr"],
    confidence=estimate["confidence"],
    sources=estimate["sources"],
)
```

Layman explanation:

The endpoint returns monthly income, confidence, source list, and supporting metrics.

Defense line:

"The Income API accepts messy alternative financial data and standardizes it into one clean monthly-income estimate."

5.12 'api/routes/loan.py'

What This File Does

This is the main full-pipeline route.

Request Inputs

```
async def apply_for_loan(
    loan_amount_npr: float = Form(...),
    sector: str = Form(...),
```

```
use_mock_income: bool = Form(False),
document: UploadFile = File(...),
):
```

Layman explanation:

This endpoint accepts both text fields and a file. That is why it uses multipart/form-data.

Input Validation

```
if loan_amount_npr <= 0:
    raise HTTPException(status_code=422, detail="loan_amount_npr must be greater than 0")
```

```
allowed_types = {"image/jpeg", "image/jpg", "image/png"}
if document.content_type not in allowed_types:
    raise HTTPException(status_code=400, detail="Document must be JPG or PNG")
```

Layman explanation:

The backend rejects invalid loan amounts and unsupported file types before running expensive processing.

SharedState Creation

```
state = SharedState(
    applicant_id=uuid.uuid4(),
    loan_amount_npr=loan_amount_npr,
    sector=sector,
)
```

Layman explanation:

This creates a new loan file folder for the request.

Parallel Execution

```
await asyncio.gather(run_document(), run_income())
```

Layman explanation:

Document OCR and income parsing run at the same time because they do not depend on each other.

Defense line:

"We run independent agents concurrently to reduce total latency. OCR can be slow, so we do not make income parsing wait unnecessarily."

Timeout Handling

```
return await asyncio.wait_for(
    doc_agent.run(state, image_bytes),
    timeout=settings.OCR_TIMEOUT_SECONDS
)
```

Layman explanation:

If OCR takes too long, the system stops waiting and marks the case for manual review.

Score Bridge

The route attempts to run Score Agent and has fallback logic:

```
try:
    state = await score_agent.run(state)
except Exception:
    base_prob = 0.5 + income_bonus + (inc_conf * 0.2) - loan_penalty
    state.credit_score = min(max(base_prob, 0.3), 0.98)
```

Layman explanation:

This was added to keep the integrated demo running even if the Score Agent method name does not match. It is a mid-defense bridge.

Professional answer:

"The route contains defensive fallback scoring during sprint integration. The cleanup is to rename or add ScoreAgent.run() and remove fallback math once the ML method contract is finalized."

Final Response

```
return LoanDecisionResponse(
    applicant_id=state.applicant_id,
    final_decision=state.final_decision or "Refer",
    decision_reason=state.decision_reason or "Processing incomplete",
    credit_score=state.credit_score,
    compliance_flags=state.compliance_flags,
)
```

Layman explanation:

The endpoint collects the final values from SharedState and returns them as JSON.

Defense line:

"This file is the orchestrator route. It wires all agents together and returns the final loan decision."

5.13 'agents/shared_state.py'

What This File Does

Defines the central data model passed through the five-agent pipeline.

Main Code Portion

```
class SharedState(BaseModel):  
    model_config = ConfigDict(validate_assignment=True)
```

Layman explanation:

This tells Pydantic to validate fields not only when the object is created, but also whenever an agent changes a value.

Example:

If `monthly_income_npr` expects a float and an agent writes "forty thousand", Pydantic can catch that.

Identity Fields

```
applicant_id: uuid.UUID  
loan_amount_npr: float  
sector: str
```

These are known at the start of the pipeline.

Document Fields

```
document_verified: Optional[bool] = None  
extracted_fields: Optional[dict] = None  
doc_confidence: Optional[float] = None  
manual_review_required: bool = False
```

These are filled by Document Agent.

Income Fields

```
monthly_income_npr: Optional[float] = None
income_confidence: Optional[float] = None
income_sources: list[str] = []
```

These are filled by Income Agent.

Score Fields

```
credit_score: Optional[float] = None
shap_explanation: Optional[list[dict]] = None
```

These are filled by Score Agent.

Compliance Fields

```
compliance_flags: list[str] = []
```

This is filled by Compliance Agent.

Decision Fields

```
final_decision: Optional[Literal["Approve", "Reject", "Refer"]] = None
decision_reason: Optional[str] = None
audit_trail_path: Optional[str] = None
```

These are filled by Decision Agent.

Important codebase note:

Some runtime code writes shap_explanations and score_confidence, but the model defines shap_explanation. This should be aligned in cleanup.

Professional answer:

"SharedState is the contract. During final hardening we will align all field names so agents and schemas use the same naming consistently."

Defense line:

"SharedState is the backbone of the whole project. It is the common language spoken by all agents."

5.14 'agents/document_agent.py'

What This File Does

Receives image bytes, runs OCR, extracts document fields, calculates confidence, and flags manual review when needed.

Main Code Portion

```
ocr_result = await run_ocr(image_bytes)
fields = extract_fields(ocr_result)
```

Layman explanation:

run_ocr reads the image and converts it to text. extract_fields tries to convert raw text into useful fields like name or citizenship number.

Handling OCR Timeout/Error

```
if ocr_result.get("timed_out"):
    state.document_verified = False
    state.doc_confidence = 0.0
    state.manual_review_required = True
    return state
```

Layman explanation:

If OCR fails, the system does not crash. It marks the document as not verified and sends it for manual review.

Confidence Calculation

```
conf_values = [
    f["confidence"] for f in fields.values()
    if f.get("confidence", 0) > 0
]
mean_confidence = round(sum(conf_values) / len(conf_values), 4)
```

Layman explanation:

Each extracted field has a confidence. The document confidence is the average confidence of found fields.

KYC Threshold

```
if mean_confidence < settings.MIN_KYC_CONFIDENCE:
    state.document_verified = False
    state.manual_review_required = True
else:
    state.document_verified = True
```

Layman explanation:

If OCR confidence is below 70 percent by default, the document is too risky for automatic acceptance.

Defense line:

"Document Agent does not claim to detect all fraud. It extracts and scores document readability. Low confidence goes to manual review."

5.15 'agents/income_agent.py'

What This File Does

Combines eSewa, remittance, and cooperative records into one monthly income estimate.

Main Code Portion

```
if esewa_data:
    esewa_signals, esewa_name = parse_esewa(esewa_data)
    all_signals.extend(esewa_signals)
```

Layman explanation:

If eSewa data exists, parse it into standard income signals.

Normalization

```
estimate = normalize_to_monthly_estimate(all_signals)
```

Layman explanation:

This combines all income signals and calculates monthly average, confidence, source count, and stability.

Name Cross-Validation

```
name_check = check_name_consistency(doc_name, income_names)
if name_check["checked"] and not name_check["is_consistent"]:
    estimate["confidence"] = round(estimate["confidence"] * 0.6, 4)
```

Layman explanation:

If the name on the document does not match income records, the system reduces income confidence by 40 percent.

Defense line:

"The Income Agent does not just add numbers. It also checks reliability using data coverage, stability, source diversity, and name consistency."

5.16 'agents/score_agent.py'

What This File Does

Loads the trained model and calculates a repayment score.

Model Loading

```
if os.path.exists(self.model_path):
    self.model = joblib.load(self.model_path)
```

Layman explanation:

The trained model is saved on disk as ml/credit_model.joblib. Score Agent loads it into memory.

Input Row

```
live_applicant_row = pd.DataFrame([
    "loan_amount_npr": loan_amount,
    "monthly_income_npr": monthly_income,
    "income_confidence": income_conf,
    "doc_confidence": doc_conf
])
```

Layman explanation:

The model expects a table row. Even for one applicant, we format data as a one-row table.

Prediction

```
probabilities = self.model.predict_proba(live_applicant_row)
repayment_probability = float(probabilities[0][1])
calculated_credit_score = int(repayment_probability * 1000)
```

Layman explanation:

The model returns probability for each class. We take the repayment class probability and convert it into a 0-1000 score.

Explanation

```
readable_narrative_list = ShapFormatter.generate_human_explanation(raw_contributions)
```

Layman explanation:

Raw mathematical impacts are converted into sentences a loan officer can understand.

Important codebase note:

The class defines `run_inference`, but the route tries to call `run`. This should be fixed by either renaming `run_inference` to `run` or adding:

```
async def run(self, shared_state):
    await self.run_inference(shared_state)
    return shared_state
```

Defense line:

"Score Agent is the ML part. It turns structured applicant features into repayment probability and human-readable risk explanations."

5.17 'agents/compliance_agent.py'

What This File Does

Checks regulatory and internal policy rules. This is not ML. It is deterministic rule logic.

Reset Flags

```
state.compliance_flags = []
```

Layman explanation:

Every compliance run starts fresh so old flags do not remain from another case.

KYC Rule

```
if state.manual_review_required:  
    state.compliance_flags.append("KYC_INCOMPLETE")
```

If document quality is low, KYC is incomplete.

Income Rule

```
income_conf = state.income_confidence or 0.0  
if income_conf < 0.25:  
    state.compliance_flags.append("INCOME_UNVERIFIABLE")
```

If income confidence is too low, the system cannot trust the income claim.

Loan-to-Asset Proxy

```
estimated_assets = monthly_income * 12 * 10  
loan_to_asset = loan_amount / estimated_assets  
if loan_to_asset > settings.MAX_LOAN_TO_ASSET:  
    state.compliance_flags.append("LOAN_TO_ASSET_BREACH")
```

Layman explanation:

For demo, asset value is estimated using annual income times 10. In production, Lalpurja valuation would be better.

Sector Exposure

```
if is_agricultural and loan_amount > settings.AGRI_SECTOR_LIMIT_NPR:  
    state.compliance_flags.append("SECTOR_EXPOSURE_LIMIT")
```

If an agriculture loan is above the configured cap, it is flagged.

AML Flag

```
if monthly_income > settings.AML_TXN_LIMIT_NPR:  
    state.compliance_flags.append("AML_FLAG")
```

Very high monthly income is flagged for review.

Defense line:

"We deliberately keep compliance rule-based because regulations are not probabilities. A rule is either passed or breached."

5.18 'agents/decision_agent.py'

What This File Does

Final stage. It reads compliance flags and credit score and produces final decision and reason.

Compliance Override

```
if state.compliance_flags:
    state.final_decision = "Refer"
    state.decision_reason = (
        f"Referred for manual review. "
        f"Compliance flags: {'; '.join(reasons)}."
    )
    return state
```

Layman explanation:

If any compliance issue exists, the system does not auto-approve.

Missing Score

```
if state.credit_score is None:
    state.final_decision = "Refer"
```

If scoring failed, send to human review.

Intended Score Logic

The intended logic is:

```
if score >= settings.APPROVE_THRESHOLD:
    final_decision = "Approve" or "Recommend"
elif score >= settings.REFER_THRESHOLD:
    final_decision = "Refer"
```



```
else:  
    final_decision = "Reject"
```

Important codebase note:

The current file has a lower block that references undefined names like `credit_score`, `APPROVE_THRESHOLD`, and `compliance_flags`. Because it is inside a `try/except`, it falls back to safe Refer on error. This is a known cleanup item.

Professional answer:

"The Decision Agent's priority design is correct: compliance first, missing score second, score threshold third. The implementation has a sprint-integration variable naming issue that we have identified and will fix before final defense."

Defense line:

"Decision Agent is intentionally conservative. When uncertain, it refers to a human rather than approving blindly."

5.19 'utils/ocr.py'

What This File Does

Handles OCR engine loading, image preprocessing, OCR execution, field extraction, and warmup.

OCR Singleton

```
_ocr_engine = None  
OCR_AVAILABLE = False
```

Layman explanation:

The OCR engine is heavy. We keep one global engine instead of creating a new one for every request.

PaddleOCR Initialization

```
_ocr_engine = PaddleOCR(  
    use_angle_cls=True,  
    lang='ne',  
    enable_mkldnn=False  
)
```

Layman explanation:

This loads PaddleOCR for Nepali/multilingual reading. `use_angle_cls=True` helps detect rotated text. `enable_mkldnn=False` is a Windows stability choice.

Image Preprocessing

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
_, binarized = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
eroded = cv2.erode(binarized, kernel, iterations=1)
```

Layman explanation:

1. Convert image to black-and-white brightness.
2. Smooth noise.
3. Convert pixels to pure black or white.
4. Thicken text slightly so OCR can detect characters better.

Defense line:

"We preprocess images before OCR because raw mobile photos often contain shadows, stamps, and noise."

Async OCR Wrapper

```
result = await asyncio.wait_for(
    loop.run_in_executor(None, _run_ocr_sync, image_bytes),
    timeout=300.0
)
```

Layman explanation:

OCR is CPU-heavy and synchronous, so it runs in a background executor while FastAPI stays async.

Field Extraction

```
flexible_pattern = r'[\u0966-\u096F]+(?:[\s\-\V]+[\u0966-\u096F]+)+'
```

```
match = re.search(flexible_pattern, raw_text)
```

Layman explanation:

This regex looks for Nepali digit patterns that may represent a citizenship number.

Mock/Fallback

```
if not OCR_AVAILABLE or engine is None:
    return {"raw_text": "", "boxes": [], "_is_mock": True}
```

Layman explanation:

If OCR engine is unavailable, the system returns a safe mock/unavailable result instead of crashing.

Important note:

This file writes debug images diagnostic_processed.jpg and ocr_input_debug.jpg. These are useful during OCR development, but production should write them only in debug mode.

5.20 'utils/income_parsers.py'

What This File Does

Converts different income data formats into one standard signal format, then calculates monthly income and confidence.

eSewa Parser

```
INCOME_TYPES = {
    "receive",
    "salary",
    "transfer_in",
    "business_income",
    "freelance",
}
```

Layman explanation:

Only money-coming-in transaction types are counted as income. Expenses are ignored.

Remittance Parser

```
amount_npr = round(amount_usd * exchange_rate, 2)
```

Layman explanation:

Foreign remittance is converted into Nepali rupees using the exchange rate.

Cooperative Parser

```
signals.append({  
    "date": date_str,  
    "amount_npr": amount,  
    "source": "cooperative",  
    "type": "regular",  
})
```

Layman explanation:

Cooperative monthly deposits are treated as regular financial capacity signals.

Normalization

```
coverage_score = min(1.0, all_months / 6)  
stability_score = 1.0 - cv  
diversity_score = min(1.0, source_count / 2)  
confidence = (  
    coverage_score * 0.45  
    + stability_score * 0.35  
    + diversity_score * 0.20  
)
```

Layman explanation:

Confidence depends on:

- How many months of data exist.
- How stable income is.
- Whether multiple sources confirm income.

Defense line:

"We do not trust income blindly. We score its reliability using coverage, stability, and source diversity."

Jaccard Name Matching

```
intersection = len(doc_tokens & income_tokens)  
union = len(doc_tokens | income_tokens)  
score = intersection / union
```

Layman explanation:

This compares overlapping words in names. It handles cases where a middle name is missing.

Defense line:

"Jaccard matching is better than exact matching for Nepali names because documents often include different name lengths or spellings."

5.21 'utils/shap_formatter.py'

What This File Does

Converts technical feature contributions into human-readable explanation sentences.

Feature Translation

```
mapping = {  
    "loan_amount_npr": "Requested Loan Amount",  
    "monthly_income_npr": "Normalized Monthly Income",  
    "income_confidence": "Income Data Reliability Score",  
    "doc_confidence": "Document Verification Confidence"  
}
```

Layman explanation:

This changes database-style names into readable labels.

Explanation Sentence

```
if impact < 0:  
    sentence = f"CRITICAL RISK: {readable_title} actively dragged down..."  
else:  
    sentence = f"POSITIVE SIGNAL: {readable_title} provided strong verification..."
```

Layman explanation:

If a factor hurts the score, it is called a risk. If it helps, it is called a positive signal.

Defense line:

"Explainability matters because loan officers and applicants need understandable reasons, not just a score."

5.22 'ml/train_model.py'

What This File Does

Creates synthetic training data and trains the XGBoost model.

Synthetic Data

```
loan_amount_npr = np.random.uniform(50000, 500000, num_samples)
monthly_income_npr = np.random.uniform(15000, 90000, num_samples)
income_confidence = np.random.uniform(0.4, 1.0, num_samples)
doc_confidence = np.random.uniform(0.5, 1.0, num_samples)
```

Layman explanation:

The script creates fake but realistic applicant profiles for prototype training.

Risk Formula

```
base_risk = (
    (loan_amount_npr / (monthly_income_npr * 12)) * 0.5
    - (income_confidence * 0.3)
    - (doc_confidence * 0.2)
)
```

Layman explanation:

Higher loan relative to income increases risk. Better income confidence and document confidence reduce risk.

Pipeline

```
credit_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("xgboost", XGBClassifier(...))
])
```

Layman explanation:

The pipeline first scales numbers, then applies XGBoost. Scaling prevents large rupee values from overwhelming small confidence values.

Save Model

```
joblib.dump(credit_pipeline, "ml/credit_model.joblib")
```

Layman explanation:

The trained model is saved so the API can load it later without retraining every time.

Defense line:

"For mid-defense, synthetic data proves the architecture. Production requires retraining and validation using real historical repayment data."

5.23 'ml/credit_model.joblib'

What This File Does

Binary saved trained model.

Defense line:

"This is the serialized ML pipeline loaded by Score Agent. It contains the scaler plus XGBoost classifier."

Do not try to open it as text during defense.

5.24 'db/models.py'

What This File Does

Defines database tables using SQLAlchemy ORM.

Enums

```
class LoanStatus(str, enum.Enum):  
    PENDING = "pending"  
    APPROVED = "approved"  
    REJECTED = "rejected"  
    REFERRED = "referred"
```

Layman explanation:

Enums restrict status values so the database does not get random strings.

Applicant Table

```
class Applicant(Base):
    __tablename__ = "applicants"
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    full_name = Column(String(255), nullable=False)
    loan_amount_npr = Column(Float, nullable=False)
    sector = Column(String(100), nullable=False)
```

Layman explanation:

Stores the main applicant and loan request information.

Document Table

```
class Document(Base):
    extracted_fields = Column(JSONB, nullable=True)
    doc_confidence = Column(Float, nullable=True)
    manual_review_required = Column(Boolean, default=False)
```

Layman explanation:

Stores OCR output and confidence. JSONB is used because extracted fields can vary by document type.

AuditLog Table

```
class AuditLog(Base):
    event_type = Column(String(100), nullable=False)
    agent_name = Column(String(100), nullable=True)
    details = Column(JSONB, nullable=True)
```

Layman explanation:

Stores a record of what happened in the system. This is important for financial auditability.

Defense line:

"Our database schema separates applicants, documents, roles, and audit logs. JSONB lets us store flexible OCR and audit details while keeping relational structure for core records."

5.25 'db/session.py'

What This File Does

Creates the database engine and session factory.

Async URL Conversion

```
_url = make_url(settings.DATABASE_URL)
ASYNC_URL = _url.set(drivename="postgresql+asyncpg")
```

Layman explanation:

This converts a normal PostgreSQL URL into an async PostgreSQL URL so FastAPI can use async database operations.

Engine

```
engine = create_async_engine(
    ASYNC_URL,
    pool_size=5,
    max_overflow=10,
    connect_args={"ssl": "require"}
)
```

Layman explanation:

The engine manages database connections. Pooling means the app reuses connections instead of opening a new one every request.

Session Factory

```
AsyncSessionLocal = sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)
```

Layman explanation:

This creates database sessions for routes.

Create Tables

```
await conn.run_sync(Base.metadata.create_all)
```

Layman explanation:

Creates missing tables at startup.

Important codebase note:

This file currently defines engine twice. The second definition overrides the first. This should be consolidated.

Professional answer:

"The active sprint has duplicate engine setup in db/session.py. The production cleanup is to keep one engine configuration that supports Supabase SSL and transaction-pooling settings consistently."

5.26 'db/migration/env.py'

What This File Does

Alembic runtime environment. It tells Alembic how to run migrations offline or online.

Main Code Portion

```
def run_migrations_online() -> None:
    connectable = engine_from_config(...)
    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)
```

Layman explanation:

When Alembic applies migrations, it connects to the database and runs schema changes.

Important note:

target_metadata = None means autogeneration is not yet connected to SQLAlchemy models. Future cleanup should set it to Base.metadata.

Defense line:

"Alembic migration scaffolding is present. For final production hardening, we will connect metadata for autogenerate and create versioned migrations."

5.27 'db/migration/README'

What This File Does

Default Alembic note saying this is a generic single-database migration setup.

5.28 'db/migration/script.py.mako'

What This File Does

Template used by Alembic when generating new migration files.

Main Code Portion

```
def upgrade() -> None:
    ${upgrades if upgrades else "pass"}

def downgrade() -> None:
    ${downgrades if downgrades else "pass"}
```

Layman explanation:

Every migration has an upgrade step and a downgrade step. Upgrade applies schema changes; downgrade reverses them.

5.29 'frontend/templates/login.html'

What This File Does

Login/portal entry page served at /.

Main Code Portion

```
<form onsubmit="event.preventDefault(); window.location.href='/dashboard';">
```

Layman explanation:

This is currently demo navigation. It does not validate real credentials yet; it redirects to

dashboard.

Tailwind CDN

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
```

Layman explanation:

Tailwind provides utility CSS classes directly in the browser for fast UI development.

Professional caveat:

"For production, frontend assets should be bundled locally and real authentication should be wired to JWT."

Defense line:

"The login page is the portal UI. Authentication backend is a planned hardening step; the current version focuses on pipeline demonstration."

5.30 'frontend/templates/apply.html'

What This File Does

Income verification console served at /apply.

Alpine State

```
x-data="{
  applicantId: '00000000-0000-0000-0000-000000000001',
  useMock: false,
  esewaRawText: "",
  incomeResponse: null,
  loading: false,
  errorMessage: "",
}"
```

Layman explanation:

This stores frontend state: applicant ID, mock mode, pasted JSON, API response, loading state, and errors.

Submit Function

```
let response = await fetch('/api/v1/income/analyze', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(payload)
});
```

Layman explanation:

The page sends JSON to the income analysis endpoint and displays the result.

Presets

The page has sample salary/agriculture JSON templates to help demo income parsing.

Defense line:

"The income console lets us test the Income Agent separately from the full pipeline, which is useful for debugging and demonstration."

5.31 'frontend/templates/dashboard.html'

What This File Does

Main underwriting dashboard served at /dashboard.

Form Fields

```
<input type="number" id="loan_amount_npr" value="150000" required>
<select id="sector">
<input type="checkbox" id="use_mock_income" checked>
<input type="file" id="document" accept="image/jpeg,image/jpg,image/png" required>
```

Layman explanation:

The user enters loan amount, selects sector, chooses mock income, and uploads a document.

FormData Request

```
const formDataPayload = new FormData();
formDataPayload.append('loan_amount_npr', loan_amount_npr);
formDataPayload.append('sector', sector);
```

```
formDataPayload.append('use_mock_income', use_mock_income);
formDataPayload.append('document', documentFile);
```

Layman explanation:

FormData is used because the request contains both text fields and a file upload.

API Call

```
const response = await fetch('http://127.0.0.1:8000/api/v1/loan/apply', {
  method: 'POST',
  body: formDataPayload
});
```

Layman explanation:

The dashboard calls the full backend loan pipeline.

Result Binding

```
document.getElementById('txtFinalDecision').innerText = data.final_decision;
document.getElementById('txtDecisionReason').innerText = data.decision_reason;
```

Layman explanation:

The returned JSON values are displayed in the UI.

Defense line:

"The dashboard is not just static UI. It calls the live FastAPI endpoint and renders actual pipeline output."

5.32 'test_ocr.py'

What This File Does

Development test for OCR behavior. It likely uploads or reads a test image and checks OCR extraction.

Defense line:

"This is a developer test helper for validating OCR behavior outside the full app."

5.33 'test_native_ocr.py'

What This File Does

Tests native PaddleOCR initialization/runtime. Useful because OCR libraries can behave differently on Windows.

Defense line:

"This helped us isolate whether OCR problems came from PaddleOCR itself or from our agent pipeline."

5.34 'test_score.html'

What This File Does

Frontend experiment/test page for score display.

Defense line:

"This was used during development to test how credit score outputs could be visualized before final dashboard integration."

5.35 'test_image.jpg'

What This File Does

Sample image used for OCR testing.

Do not treat it as source code.

5.36 'diagnostic_processed.jpg' and 'ocr_input_debug.jpg'

What These Files Do

Generated debug images from OCR preprocessing.

Defense line:

"These are debug artifacts that show what the OCR pipeline sees after preprocessing. They are useful for improving OCR accuracy."

5.37 'fix_ocr.py', 'fix_income_agent.py', 'fix_files.py', 'fix_compliance_decision.py'

What These Files Do

These are development repair scripts used while stabilizing the project. They are not core runtime modules.

How to explain professionally:

"During sprint work, we used helper scripts to patch or test specific modules quickly. The final runtime path is through main.py, api/routes, agents, utils, db, and frontend/templates. The fix_*.py files are development artifacts and can be archived after final cleanup."

Do not present these as production architecture.

5.38 'docs/reference.pdf'

What This File Does

Reference handbook PDF used as formatting/style inspiration.

5.39 'docs/mid_defense_handbook.md'

What This File Does

This editable handbook source. If the team wants changes, edit this file first.

5.40 'docs/generate_handbook_pdf.py'

What This File Does

Local PDF generator that converts this Markdown handbook into a PDF without external packages.

Main Code Portion

```
SOURCE = ROOT / "mid_defense_handbook.md"  
OUTPUT = ROOT / "RinSathi_Mid_Defense_Handbook.pdf"
```

Layman explanation:

It reads the Markdown file and writes the final PDF.

PDF Building

```
pdf.extend(b"%PDF-1.4\n")
```

Layman explanation:

This script manually creates a simple PDF file. It was used because PDF libraries were not installed in the environment.

Defense line:

"This is documentation tooling, not part of the product runtime."

5.41 Generated PDF Files In 'docs/'

RinSathi_Mid_Defense_Handbook.pdf is the exported final handbook.

If asked:

"The PDF is generated from the Markdown handbook so we can maintain one editable source and regenerate a final shareable file."

5.42 '.gitignore'

What This File Does

Tells Git which files/folders to ignore, such as environment folders, cache files, and secrets.

Defense line:

".gitignore prevents generated files and sensitive/local environment files from polluting version control."

5.43 '.git/'

What This Folder Does

Stores Git history and metadata. Not application code.

5.44 'venv/'

What This Folder Does

Python virtual environment containing installed dependencies.

Defense line:

"venv isolates dependencies so this project does not interfere with other Python projects on the same computer."

Do not discuss internal package files inside venv; they are third-party library files.

5.45 '__pycache__/'

What This Folder Does

Python-generated compiled cache files.

Defense line:

"__pycache__ improves Python import performance. It is generated automatically and is not handwritten code."

6. Swagger Docs: What To Show And What To Say

Swagger URL:

<http://127.0.0.1:8000/docs>

What Swagger Is

Swagger is an interactive API documentation page automatically generated by FastAPI.

It is built from:

- Route decorators like `@router.post(...)`.
- Pydantic request models.
- Pydantic response models.
- Endpoint tags, summaries, and descriptions.

Why It Matters

Swagger proves that the backend is not a hidden black box. Anyone can inspect endpoints, see required inputs, test requests, and view responses directly.

What To Demo

1. Open `/docs`.
2. Expand GET `/api/v1/income/mock-data`.
3. Click "Try it out".
4. Click "Execute".
5. Show response JSON.
6. Expand POST `/api/v1/income/analyze`.
7. Show request schema.
8. Explain that frontend developers can use this as API contract.
9. Expand POST `/api/v1/loan/apply`.
10. Show that it accepts form data and file upload.

Defense Answer

"Swagger is our live API contract. It improves collaboration because frontend developers, testers, and evaluators can test endpoints without reading backend code. Since it is generated from FastAPI and Pydantic, it stays close to the actual implementation."

If Asked Why Some HTML Pages Are Not In Swagger

"Swagger is mainly for APIs, not web pages. Some frontend routes are hidden using `include_in_schema=False` because they serve HTML, not JSON API contracts."

7. Important Engineering Concepts In Layman Terms

FastAPI

Python framework for building APIs quickly. It receives HTTP requests and returns JSON or HTML.

Endpoint

A URL function. Example: `/api/v1/loan/apply` is an endpoint that receives loan applications.

Router

A group of related endpoints. `income.py` groups income endpoints; `loan.py` groups loan endpoints.

Pydantic

Data validation tool. It checks whether data has the right shape and type.

SharedState

Central loan file folder passed through all agents.

Async/Await

Lets slow tasks run without freezing the whole server.

`'asyncio.gather'`

Runs multiple async tasks concurrently.

OCR

Optical Character Recognition. It reads text from images.

OpenCV

Image-processing library used to clean document images before OCR.

XGBoost

Machine learning algorithm good for table-like data.

SHAP / Explainability

Explains which input factors helped or hurt the score.

Compliance Agent

Rule checklist. It ensures risky cases are not auto-approved.

Refer

Send to human review. It is a safety mechanism, not an error.

JSONB

PostgreSQL type for storing JSON in a searchable way.

UUID

Unique identifier. Safer than simple numbers for financial records.

8. Known Limitations And Honest Professional Answers

Limitation 1: Synthetic ML Data

The model is trained on synthetic data.

Answer:

"For mid-defense, synthetic data demonstrates the architecture and feature pipeline. A production system must retrain on real historical MFI repayment data and be validated for accuracy and fairness."

Limitation 2: OCR Accuracy

OCR may fail on low-quality Nepali document scans.

Answer:

"We handle this safely using confidence thresholds. Low-confidence OCR does not auto-approve; it routes to manual review."

Limitation 3: Score Agent Method Mismatch

ScoreAgent defines `run_inference`, while route code tries `run`.

Answer:

"This is a sprint integration naming issue. The design is clear, and the route currently has fallback logic. Before final defense, we will align the method contract."

Limitation 4: Decision Agent Variable Naming Issue

Some variables in `decision_agent.py` should use state and settings.

Answer:

"The intended priority logic is correct: compliance first, missing score second, thresholds third. We have identified a variable naming issue in implementation and will fix it in final hardening."

Limitation 5: Duplicate Income Route

`/api/v1/income/analyze` exists in both `main.py` and `api/routes/income.py`.

Answer:

"This came from sprint integration. We will keep the modular route file version and remove the duplicate from `main.py`."

Limitation 6: Duplicate DB Engine

db/session.py defines engine twice.

Answer:

"The final active engine works, but we will consolidate it into one clean configuration for Supabase."

Limitation 7: Authentication Not Fully Wired

Login UI exists, JWT config exists, but route protection is not complete.

Answer:

"Authentication is planned for the next sprint. Mid-defense focuses on proving the loan pipeline and API architecture."

Limitation 8: In-Memory OCR Jobs

Document job status is stored in memory.

Answer:

"This is acceptable for demo, but production should move job tracking to Redis or PostgreSQL so status survives server restart."

9. Live Demo Script

Demo Preparation

Run:

```
.\.venv\Scripts\Activate.ps1  
uvicorn main:app --reload
```

Open:

<http://127.0.0.1:8000/>

Demo Flow

1. Show login page.
2. Click dashboard.
3. Explain loan amount, sector, mock income toggle, and document upload.
4. Upload sample image if available.
5. Submit application.
6. Explain pipeline progress:
 - Document and Income run together.
 - Score, Compliance, Decision run after.
7. Show final decision.
8. Open /docs.
9. Test /income/mock-data.
10. Show /loan/apply request schema.

What To Say During Demo

"The dashboard calls the live backend endpoint. The backend creates SharedState, runs agents, checks compliance, and returns JSON. The UI simply visualizes that response."

10. Defense Q&A Bank

Q: What is the main innovation?

A: The main innovation is combining alternative income signals with OCR and compliance guardrails in one structured multi-agent pipeline for rural microfinance.

Q: Why not just use a normal loan form?

A: A normal form only collects user-entered data. RinSathi processes documents, normalizes financial records, scores risk, checks rules, and explains the result.

Q: What is SharedState?

A: SharedState is a Pydantic model that acts like a loan file folder passed through all agents. Each agent writes its result into it.

Q: Why use Pydantic?

A: It validates data types and shapes, preventing incorrect data from moving silently through the financial decision pipeline.

Q: Why use OCR?

A: Rural loan documents often exist as physical images. OCR converts those images into text that the system can process.

Q: Can OCR detect fake documents?

A: Not by itself. OCR extracts fields and confidence. Fraud resistance comes from cross-checking document names with income records and routing low-confidence cases to manual review.

Q: Why use alternative income?

A: Many rural borrowers do not have formal salary slips but still have income through wallets, remittances, cooperatives, and small businesses.

Q: Why use XGBoost?

A: XGBoost works well for tabular data and is widely used for risk scoring because it is fast, accurate, and explainable with feature contribution techniques.

Q: Why not neural networks?

A: Neural networks are better for images, text, or huge datasets. Our risk features are tabular, so XGBoost is more practical and easier to explain.

Q: What is SHAP?

A: SHAP explains how each feature contributed to the model output. It helps convert black-box scores into understandable reasons.

Q: Why does compliance override ML?

A: Regulations are mandatory. A high ML score cannot override KYC or policy violations.

Q: What does "Refer" mean?

A: Refer means the system needs a human loan officer to review the case. It is used for uncertain, incomplete, or compliance-sensitive cases.

Q: Why use Swagger?

A: Swagger documents and tests the API automatically. It proves the backend endpoints are structured and usable beyond the frontend.

Q: What database are you using?

A: The project is designed for Supabase PostgreSQL using SQLAlchemy async access.

Q: What would you improve before final defense?

A: Clean duplicate routes and DB engine setup, align Score Agent method names, fix Decision Agent variable references, add JWT route protection, persist audit logs, and add automated tests.

11. Team Study Plan

Everyone Must Know

- One-line pitch.
- Five agents in order.
- What SharedState means.
- What /api/v1/loan/apply does.
- How Swagger works.
- Why compliance overrides ML.
- Known limitations and honest answers.

Backend Presenter Must Know

- main.py
- api/routes/loan.py
- agents/shared_state.py
- all agents/*.py
- utils/ocr.py
- utils/income_parsers.py
- db/session.py
- db/models.py

Frontend Presenter Must Know

- login.html
- dashboard.html

- apply.html
- Fetch API calls.
- FormData for file upload.
- How dashboard displays JSON response.

Business Presenter Must Know

- Rural lending problem.
- Alternative income importance.
- Nepal microfinance context.
- Why human review remains important.

Testing Presenter Must Know

- OCR tests.
- Income mock-data endpoint.
- Swagger testing.
- Edge cases: blurry document, no income, high loan, agriculture cap, missing score.

12. Final Professional Closing

Use this when wrapping up:

"RinSathi demonstrates a practical path toward faster and more inclusive rural credit assessment. The project combines OCR, alternative data, machine learning, compliance rules, and explainability in a modular FastAPI pipeline. For mid-defense, our focus is proving the architecture and working flow. Before final defense, our focus will be hardening: cleaning integration issues, improving persistence, adding authentication, adding tests, and preparing production-grade audit trails."

One-sentence final line:

"RinSathi is not just a loan form; it is an explainable, compliance-aware underwriting assistant for Nepal's microfinance context."